Universitas Esa Unggul
Powered by Arizona State University®

CSF308

OBJECT ORIENTED PROGRAMMING

Sesi 7

Memory Management





Dosen Koordinator:

Dewi Setiowati, A.Md., S.Pd., M.Tr.Kom.

NID:

8413

E-mail:

dewi.setiowati@esaunggul.ac.id

VISI

Menjadi perguruan tinggi kelas dunia berbasis intelektualitas, kreatifitas dan kewirausahaan, yang unggul dalam mutu pengelolaan dan hasil pelaksanaan Tridarma Perguruan Tinggi.

MISI

1. Menyelenggarakan pendidikan tinggi yang bermutu dan relevan
2. Menciptakan suasana akademik yang kondusif
3. Memberikan pelayanan prima kepada seluruh pemangku kepentingan

College Contract

1. Students are required to attend or join the class on time with a maximum **delay of 15 minutes**.
2. Every student's class is required to do a presence **scan QR Code on SIAKAD**
3. Students should attend at least 80% of the scheduled lectures to be able to take the Final test.

Assessment

Coursework evaluation will be weighted as follows:

➢ Mid-Semester Examination (T 30% + P 70%)	30%
➢ Final Examination (T 30% + P 70%)	40 %
➢ Others (class participation 10%, quiz 10%, practical assignments 10%)	30 %
Total	100%

Topik Sebelum UTS

Sesi 1

Introduction to Object Oriented Programming

Sesi 2

Data Type, Variable, Method and Expression

Sesi 3

Control Statement

Sesi 4

Function and Parameter

Sesi 5

Encapsulation (Concept)

Sesi 6

Implementation Encapsulations In Case Study

Sesi 7

Memory Management

Ujian Tengah Semester

Topik Sebelum UAS

Sesi 8

Inheritance Basic

Sesi 9

Inheritance Details

(Case Study)

Sesi 10

Polymorphism

Sesi 11

Exception Handling

Sesi 12

Standard Template Library

Sesi 13

Class Presentation

Sesi 14

Class Presentation and Cource Review

Ujian Akhir Semester

Capaian Pembelajaran

Setelah mempelajari materi pada pertemuan ini, diharapkan mahasiswa :

1. Mampu memahami *Memory Management*
2. Mampu membuat dan mengimplementasikan *Memory Management* ke dalam sebuah aplikasi dalam Pemrograman Java (NetBeans)

Pendahuluan

Memory Management adalah tempat menyimpan jenis **memori** stack atau data-data, variable, dan object yang bersifat static.

Method Static adalah memungkinkan untuk dipanggil melalui nama kelas secara langsung ataupun melalui objek.

Variable Static adalah Variabel dikelas yang di deklarasikan diluar metode yang diawali kata-tercadang static akan membuat variabel tersebut menjadi variabel kelas sehinggga semua objek akan mengacu variabel ini secara bersama-sama



MEMORY ALLOCATION IN JAVA

Memory Allocation in Java

Ketika suatu program dijalankan, area memori terpisah dialokasikan untuk masing-masing program.

- **Class**
- **Interface**
- **Object**
- **Running method**

Memory Allocation in Java

- **Call stack / runtime stack**

  Digunakan untuk informasi *method* saat *method* sedang dijalankan
  - **Local variables**
  - **Formal parameters**
  - **Return value**
  - **Where method should return to**

- **Heap**

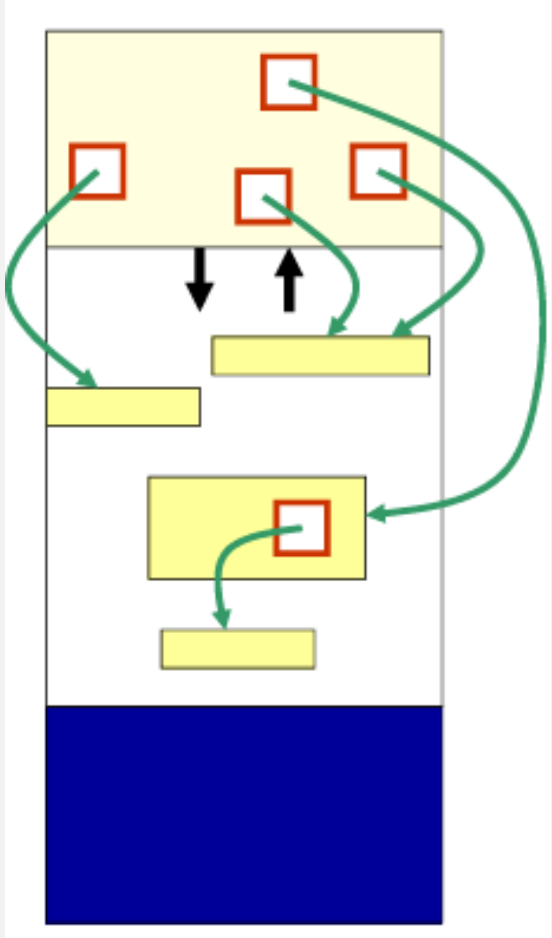
  Digunakan untuk
  - **Static information (interfaces and classes)**
  - **Instance information (objects)**

Memory Allocated to your program

call stack

objects in the heap

static space in the heap



Memory Allocation in Java

❖ **Example: What happens when an object is created by new, as in**

Person friend = new Person(…);

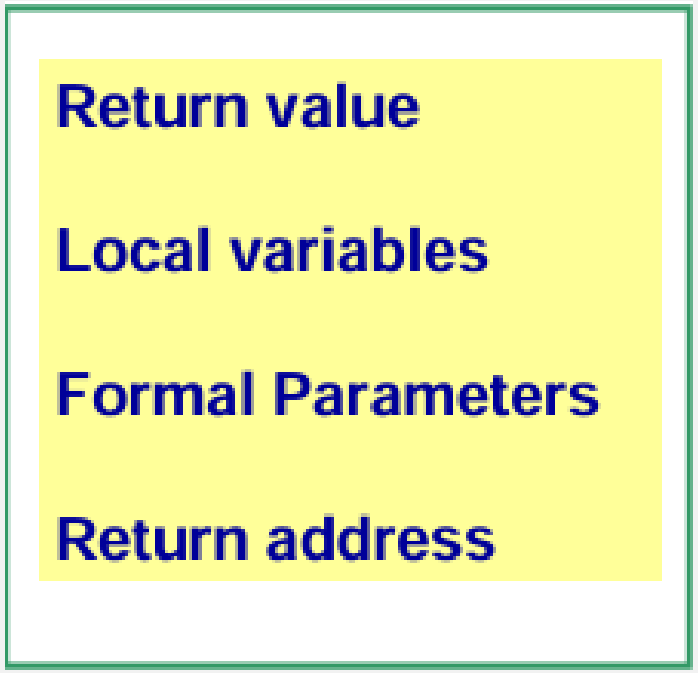
- Variabel referensi mempunyai memori yang dialokasikan padanya di tumpukan panggilan (**call stack).**
- Objek dibuat menggunakan memori di tumpukan (**heap).**

Runtime Stack

❖ **Call stack / runtime stack**
adalah ruang memori yang digunakan untuk *method* informasi saat suatu *method* sedang dijalankan.

- Ketika suatu metode dipanggil, bingkai panggilan (atau catatan aktivasi) untuk metode tersebut dibuat dan “pushed” ke tumpukan panggilan (call stack).
- Semua informasi yang diperlukan selama eksekusi *method* dikelompokkan bersama dalam bingkai panggilan

Call Frame (Activation Record for a Method)



Call Frame (Activation Record)

- **A call frame contains :**
  - Address to return to after method ends
  - Method's formal parameter variables
  - Method's local variables
  - Return value (if any)

Note :

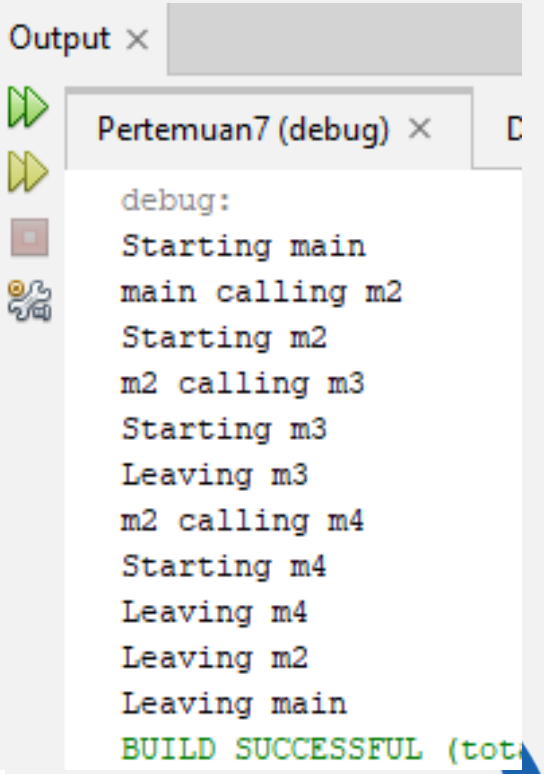
Nilai-nilai dalam **call frame** hanya dapat diakses ketika method terkait sedang dijalankan.

Example : a Typical Calling Sequence

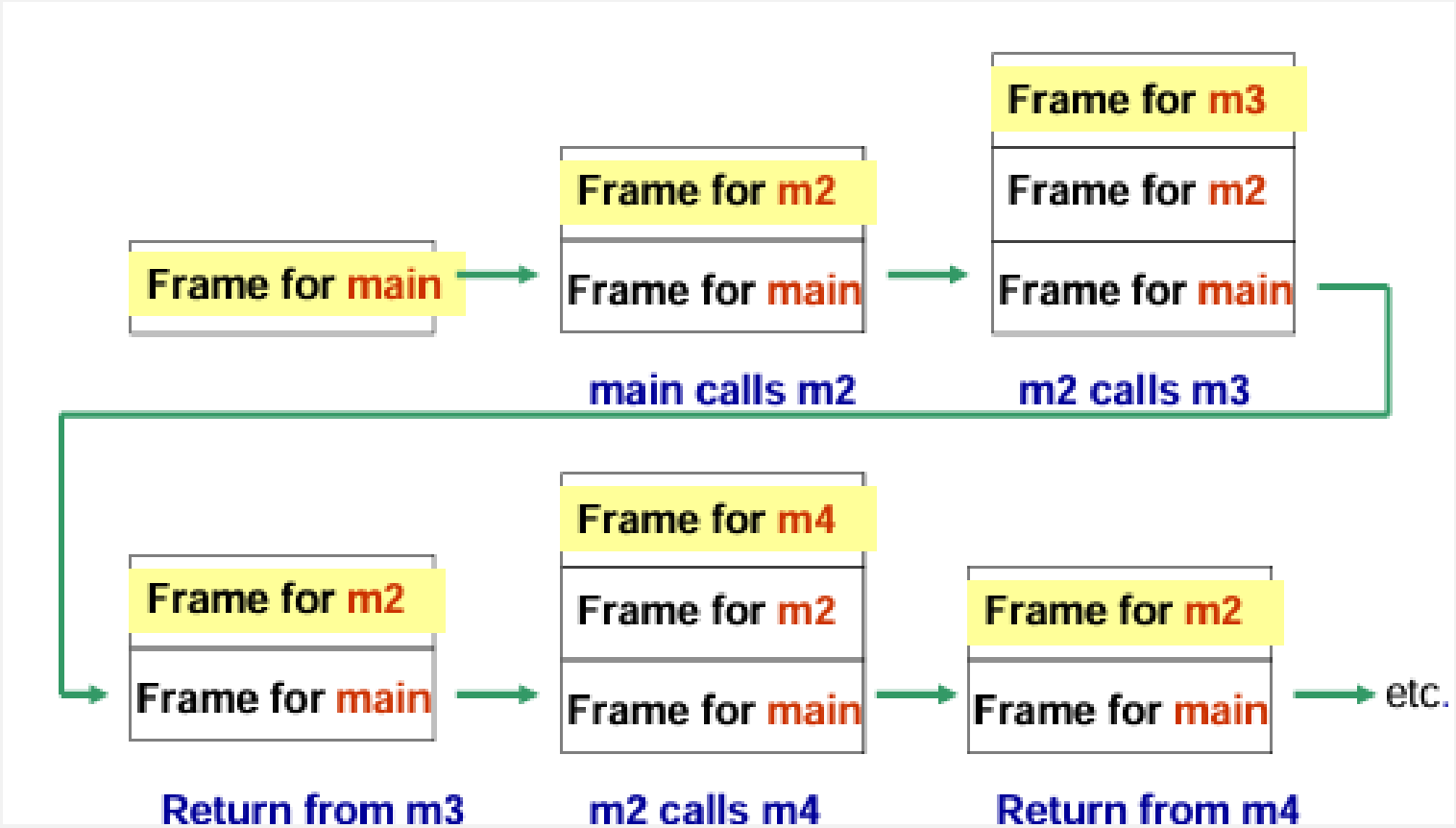
```
public class CallStackDemo {
    public static void m2()
    {
    System.out.println("Starting m2");
    System.out.println("m2 calling m3");
    m3();
    System.out.println("m2 calling m4");
    m4();
    System.out.println("Leaving m2");
    return;
}
 public static void m3()
 {
    System.out.println("Starting m3");
    System.out.println("Leaving m3");
    return;
 }
```

```
  public static void m4()
  {
     System.out.println("Starting m4");
     System.out.println("Leaving m4");
     return;
  }

  public static void main(String args[])
  {
  System.out.println("Starting main");
  System.out.println("main calling m2");
  m2();
  System.out.println("Leaving main");
  }
}
```



Call Stack for a Typical Calling Sequence



Call Stack for a Typical Calling Sequence

Ketika main method dipanggil:

- **Bingkai panggilan untuk main dibuat dan dimasukkan ke tumpukan runtime**

Ketika main memanggil method m2:

- **Bingkai panggilan untuk m2 dibuat dan dimasukkan ke tumpukan runtime**

Ketika m2 memanggil m3:

- **Bingkai panggilan untuk m3 dibuat dan dimasukkan ke tumpukan runtime**

- **Ketika m3 berakhir, bingkai panggilannya dihilangkan dan kontrol kembali ke m2**

Call Stack for a Typical Calling Sequence

Ketika m2 sekarang memanggil m4:

- **Apa yang terjadi selanjutnya?**
- **Apa yang terjadi jika m4 dihentikan?**

- **Apa yang terjadi bila m2 berakhir?**

- **Apa yang terjadi bila main berakhir? Call frame dihilangkan dan kontrol kembali ke sistem operasi**

Call Frames

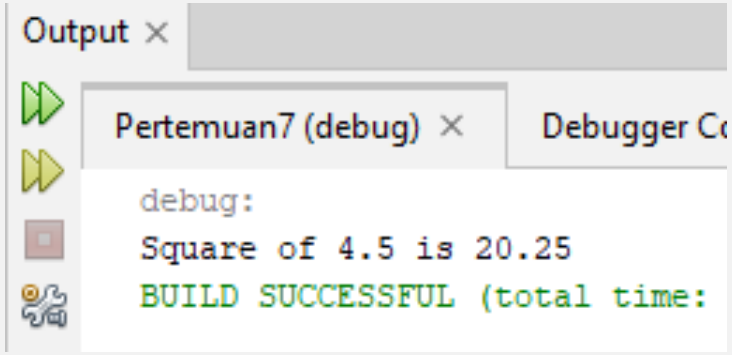
Sekarang kita akan melihat beberapa contoh apa yang ada dalam call frame untuk suatu method

- **Pertama untuk variabel sederhana**
- **Kemudian untuk variabel referensi**

Example : Call Frames – Simple Variables

```
public class CallFrameDemo1 {
  public static double square(double n){
    double temp;
    temp = n*n;
    return temp;
    }
  public static void main(String args[]){
    double x = 4.5;
    double y;
    y=square(x);
    System.out.println("Square of "+x+" is " +y);

  }
}
```



Call Frames – Example 1

Buatlah gambar frame panggilan pada tumpukan panggilan (call stack):

- **Apa yang akan ada dalam frame panggilan untuk main method ?**
  - **Address to return to in operating system**
  - **Variable args**
  - **Variable x**
  - **Variable y**

- **Apa yang akan ada dalam bingkai panggilan untuk metode persegi?**
  - **Address to return to in main**
  - **Variable n**
  - **Variable temp**
  - **Return value**

Task

- Akan ada bingkai panggilan (call frame) pada tumpukan panggilan (call stack) untuk setiap method yang dipanggil. Jadi frame panggilan apa lagi yang akan dimasukkan ke tumpukan panggilan (call frame) untuk contoh diprogram kalian?

- Bingkai panggilan manakah yang akan berada di tumpukan panggilan pada saat yang sama?

Heap Space

- **Static space :**
  berisi satu salinan dari setiap kelas dan antarmuka yang diberi nama dalam program
  - **Berisi variabel statis dan method**

- **Object space**
  - **Informasi disimpan tentang setiap objek:**
    - **Nilai variabel instannya**
    - **Jenis objek (yaitu nama kelas)**

Object Creation

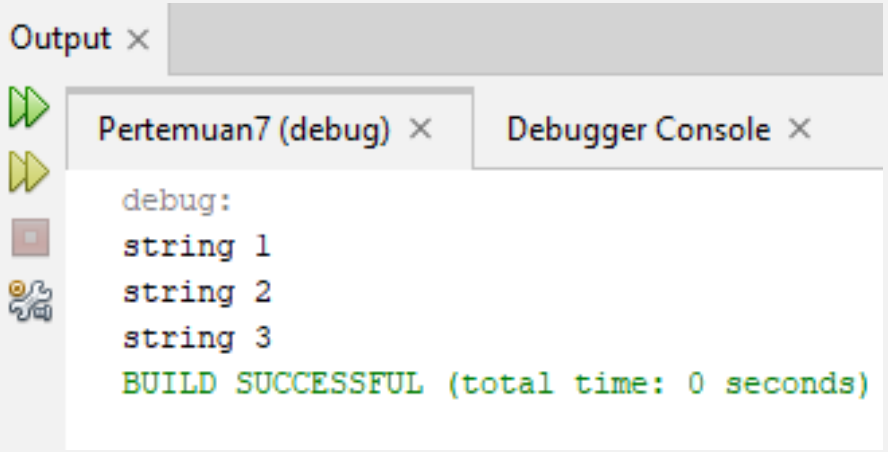
- Sekarang mari kita lihat variabel referensi…
- Memori dialokasikan di area heap ketika suatu objek dibuat menggunakan yang baru
- Variabel referensi dimasukkan ke dalam frame panggilan pada runtime stack
- Objek dibuat menggunakan memori di heap

Example : Call Frames – Reference Variables

```
public class CallFrameDemo2 {
    private static void printAll(String s1, String s2, String s3){
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
    }
    public static void main(String args[]){
        String str1, str2, str3;
        str1 = new String("string 1");
        str2 = new String("string 2");
        str3 = new String("string 3");

        printAll(str1, str2, str3);
    }

}
```



Call Frames – Example 2

- Buatlah gambar tumpukan panggilan dan heap saat program dijalankan
- Apa yang akan menjadi urutan call frame pada tumpukan panggilan?

for main
    for String constructor for str1 – then popped off
    for String constructor for str2 – then popped off
    for String constructor for str3 – then popped off
    for printAll
    for toString for str1 – then popped off
    for System.out.println – then popped off
etc.

Call Frames – Example 2

Apa yang akan ada di bingkai panggilan untuk main? (dan ditumpukan/heap?)

- Address to return to in operating system
- Variable args
- Variable str1
  - Initially?
  - After return from String constructor?
- Variable str2
- Variable str3

Apa yang akan ada dalam bingkai panggilan (call frame) untuk printAll? Panggilan

Memory Deallocation

Apa yang terjadi ketika suatu method returns?

- **Pada tumpukan runtime:**
  call frame otomatis muncul ketika method returns

- **Jadi, memori tersebut tidak dialokasikan (deallocation)**

Memory Deallocation

- Apa yang terjadi pada objek di heap?
  Sebuah objek tetap berada di tumpukan meskipun ada tidak lagi menjadi variabel yang mereferensikannya!

- Jadi, Java mempunyai secara otomatis sampah koleksi (garbage collection)
  - Secara teratur mengidentifikasi objek yang tidak lagi memiliki referensi variabel mereka, dan membatalkan alokasi memori itu (deallocates)

contoh

```
public class MemoryManagementDemo {


   public static void main(String[] args) {
      System.out.println("=== A Typical Calling Sequence ===");
      int result = addNumbers(5, 10);
      System.out.println("Result: " + result);
      System.out.println("\n=== Call Frames – Simple Variables ===");
      simpleVariables();
      System.out.println("\n=== Call Frames – Reference Variables ===");
      referenceVariables();
   }
   // a. Typical Calling Sequence
   static int addNumbers(int a, int b) {
      int sum = a + b;
      return sum;
   }
```

```
   // b. Call Frames – Simple Variables
   static void simpleVariables() {
         int x = 10;   // primitive type (stored in stack)
         int y = 20;
         int z = x + y;
         System.out.println("x = " + x + ", y = " + y + ", z = " + z);
      }
      // c. Call Frames – Reference Variables
      static void referenceVariables() {
         int[] numbers = new int[3];  // reference variable in stack, array in heap
         numbers[0] = 10;
         numbers[1] = 20;
         numbers[2] = 30;
        for (int n : numbers) {
            System.out.println("Number: " + n);
         }
         // Create object in heap
         Person person = new Person("Dina", 22);
         System.out.println(person);
      }
   }
```

```
// Class Person disimpan di heap saat diinisialisasi
class Person {
  String name;  // reference variable ke String di heap
   int age;     // simple variable

  Person(String name, int age) {
    this.name = name;
     this.age = age;
   }

   public String toString() {
     return "Person{name='" + name + "', age=" + age + "}";
   }
}
```

Output

```
=== A Typical Calling Sequence ===
Result: 15

=== Call Frames ? Simple Variables ===
x = 10, y = 20, z = 30

=== Call Frames ? Reference Variables ===
Number: 10
Number: 20
Number: 30
Person{name='Dina', age=22}

=== Code Execution Successful ===
```

TASK

Implementasikan **memory management** dalam program yang telah Anda buat masing” dengan menggunakan :

a. A typical calling sequence

b. Call Frames – Simple Variables

c. Call Frames – Reference Variables

Universitas Esa Unggul
Powered by Arizona State University®

Terima Kasih

